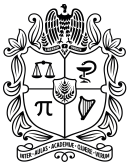
Manual técnico y manual de usuario

Taller No 01 Simulación de MANET

Modelos estocásticos y simulación en computación y comunicaciones

Autores:
David Fernando Benjumea Mora
Daniel Alonso Gracia Pinto
Stephan Alain Roland García Martiquet
José Esteban Alvarado Monroy
Óscar Iván Ulises Gutiérrez Palacios
John Jairo Páez Albino

2026-2S
Profesor: Jorge Eduardo Ortiz Triviño



UNIVERSIDAD
NACIONAL
DE COLOMBIA

4 de noviembre de 2026

Índice

1 Manual técnico **2**
1.1 Requisitos del sistema . . . 2
1.2 Programas y librerías necesarios . . . 2
1.3 Archivo fuente . . . 2
1.4 Arquitectura del programa . . . 2
1.5 Configuración de red y movilidad . . . 3
1.6 Tráfico y métricas . . . 3
1.7 Ejecución del programa . . . 3

2 Manual de usuario **4**
2.1 Antes de empezar . . . 4
2.2 Paso 1: correr una simulación . . . 5
2.3 Paso 2 (opcional): comprobar que la red está conectada . . . 6
2.4 Paso 3: ejecutar los escenarios . . . 6
2.5 Paso 4: interpretar la salida . . . 6
2.6 Visualizar una simulación con NetAnim . . . 6
2.7 Problemas frecuentes . . . 7

1 Manual técnico

Este manual describe la solución desde el punto de vista de su construcción y de su funcionamiento interno. Está dirigido a quien necesite instalar el entorno, comprender cómo está organizado el programa y reproducir los experimentos o extenderlo.

1.1 Requisitos del sistema

El entorno de referencia del grupo es Ubuntu 22.04 LTS, en ejecución nativa o mediante WSL2 sobre Windows, que es la configuración con la que se documentó la instalación y se corrieron los experimentos. El programa no depende de esa distribución en particular: también puede ejecutarse en Windows nativo o en otras distribuciones de Linux, siempre que se disponga de los programas y librerías que se indican a continuación.

1.2 Programas y librerías necesarios

Se requiere lo siguiente:

- ns-3.43, con los módulos `wifi`, `aodv`, `mobility`, `internet`, `applications` y `flow-monitor`. El módulo `netanim` es opcional y solo se usa para las trazas de visualización.
- Compilador de C++ compatible con C++17: GCC 11 o superior, Clang, o MSVC en Windows.
- CMake 3.22 o superior y Ninja.
- Python 3.10 o superior, requerido por el lanzador de ns-3.
- `wget` o `curl` con `bzip2` para descargar y descomprimir el paquete de ns-3.

La instalación de ns-3 se hace con la guía oficial de la plataforma elegida; una vez instalado, el lanzador de ns-3 compila y ejecuta el programa con CMake y Ninja.

1.3 Archivo fuente

La simulación se entrega en un único archivo fuente, `manet-jerarquica.cc`. ns-3 solo compila los programas ubicados en su carpeta `scratch/`, así que ese archivo se copia allí antes de ejecutar. Cada corrida escribe su XML de FlowMonitor y su resumen en la carpeta de salida que se indique.

1.4 Arquitectura del programa

El programa está organizado en dos estructuras de datos y un conjunto de funciones, cada una responsable de una fase de la simulación. La estructura `Parametros` reúne todas las variables configurables por línea de comandos, desde el nombre del escenario y las velocidades de movilidad hasta los alcances de radio y el directorio de salida. La estructura `RedJerarquica` actúa como la interfaz compartida entre las fases: guarda los contenedores de nodos agrupados por clúster y superclúster, la lista de *cluster heads*, la de miembros y las interfaces IP de cada subred.

Las funciones siguen el orden natural de construcción de la red. `CrearNodos` instancia los veinte nodos y los agrupa; `ConfigurarRed` instala los canales WiFi, la pila IP y el protocolo AODV, y asigna las subredes; `ConfigurarMovilidad` monta la movilidad jerárquica en tres capas; `ConfigurarTrafico` inyecta los ocho flujos UDP; y `ConfigurarMetricas` instala el monitor de flujos. Al terminar la ejecución, `GuardarMetricas` serializa los resultados. La figura 1 muestra la red que construyen esas funciones y la figura 2 el orden en que se invocan.

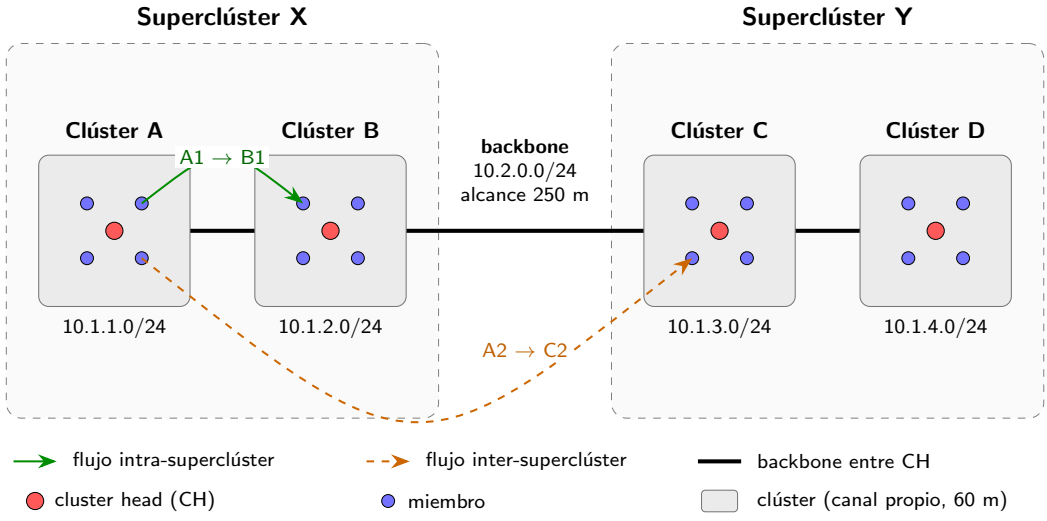


Figura 1: Arquitectura jerárquica construida por el programa: cuatro clústeres con su canal propio y un *backbone* compartido entre los *cluster heads*.

1.5 Configuración de red y movilidad

La red separa la comunicación local de la comunicación interclúster. Cada clúster usa un canal WiFi ad hoc independiente con un alcance efectivo de 60 m, y los cuatro *cluster heads* comparten un canal *backbone* con un alcance de 250 m. La propagación se modela con un disco de alcance exacto, de modo que un receptor recibe la señal si y solo si su distancia al transmisor no supera el alcance configurado. El direccionamiento asigna una subred por clúster (10.1.1.0/24 a 10.1.4.0/24) y una para el *backbone* (10.2.0.0/24), y AODV corre sobre todas las interfaces.

La movilidad se compone con tres capas anidadas. El superclúster permanece fijo; el clúster se mueve con un modelo *Random Waypoint* de velocidad constante dentro de una región de ±150 m; y cada miembro se mueve con una caminata aleatoria bidimensional dentro de una región de ±40 m. Poner la velocidad de una capa en cero equivale a dejarla quieta, que es exactamente como se aíslan los efectos en los escenarios del experimento. Las velocidades son constantes, no un rango, para que los escenarios sean comparables entre sí.

1.6 Tráfico y métricas

El tráfico consiste en ocho flujos UDP de tasa constante, con paquetes de 512 bytes a 4 paquetes por segundo (16 kbit/s por flujo). Cuatro flujos comunican miembros de clústeres del mismo superclúster y cuatro comunican miembros de superclústeres distintos, de forma que todos los paquetes cruzan la jerarquía. Los flujos inician con un desfase uniforme entre los 10 y los 11 segundos, para dejar que AODV estabilice sus rutas, y terminan a los 195 segundos. Las métricas se obtienen con el módulo `flow-monitor`, que por cada flujo contabiliza paquetes transmitidos y recibidos, bytes, retardos y variación de retardo; a partir de esos contadores se calculan la tasa de entrega, el retardo medio, el *throughput* y el *jitter*. El resultado se guarda en un XML cuyo nombre codifica el escenario, las velocidades y la réplica, con el formato `<escenario>_cs<clusterSpeed>_ns<nodeSpeed>_run<RngRun>.xml`.

1.7 Ejecución del programa

Para ejecutar una simulación se copia el programa a la carpeta `scratch/` de ns-3 y se invoca el lanzador con los parámetros deseados:

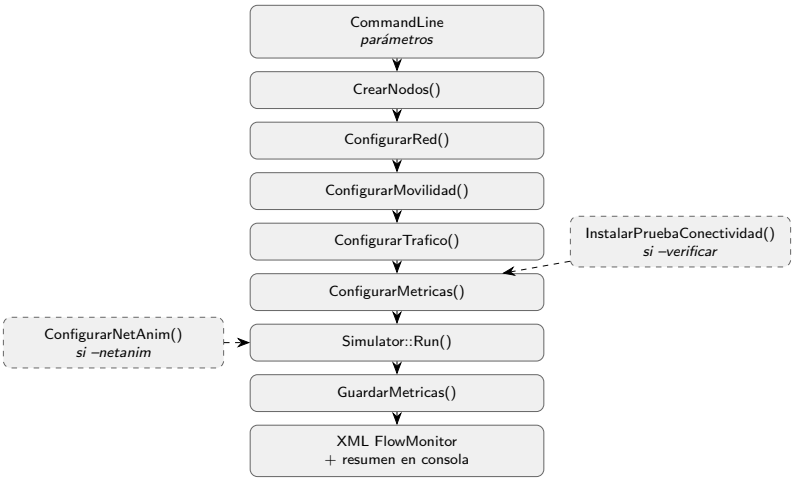


Figura 2: Secuencia de fases de la simulación y correspondencia con las funciones del programa.

```
cp manet-jerarquica.cc \
    ~/ns-allinone-3.43/ns-3.43/scratch/
cd ~/ns-allinone-3.43/ns-3.43
./ns3 run "scratch/manet-jerarquica --scenario=E3 --clusterSpeed=5 \
    --nodeSpeed=5 --simTime=200 --RngRun=1"
```

La primera ejecución compila el programa; las siguientes lo reutilizan. El nombre del escenario es solo una etiqueta: lo que determina qué se mueve son las velocidades. El programa imprime al inicio la topología y, al final, un resumen con los totales agregados. Ese resumen incluye también el tráfico de control de AODV, por lo que las cifras que se calculan sobre los flujos de datos únicamente no coinciden con las de la consola.

2 Manual de usuario

Este manual explica cómo ejecutar la simulación propuesta, cómo leer su salida y cómo visualizar el movimiento de los nodos. La instalación de ns-3 y la estructura del código están en el manual técnico.

2.1 Antes de empezar

Se necesita:

- Un sistema Linux (por ejemplo, Ubuntu 22.04), WSL2 sobre Windows o Windows nativo, con las dependencias de ns-3 instaladas (ver el manual técnico).
- ns-3.43 compilado con los módulos `wifi`, `aodv`, `mobility`, `internet`, `applications`, `flow-monitor` y `netanim` (ver el manual técnico).
- Python 3.10 o superior, requerido por el lanzador de ns-3.

En los ejemplos se supone que ns-3 está en `~/ns-allinone-3.43/ns-3.43` y que `manet-jerarquica.cc` está en la carpeta actual; si ns-3 está en otro lugar, basta con cambiar esa ruta.

2.2 Paso 1: correr una simulación

El programa es el archivo `manet-jerarquica.cc`. ns-3 compila los programas de su carpeta `scratch/`, así que primero se copia allí y luego se ejecuta desde la carpeta de ns-3:

```
cp manet-jerarquica.cc ~/ns-allinone-3.43/ns-3.43/scratch/
cd ~/ns-allinone-3.43/ns-3.43
./ns3 run "scratch/manet-jerarquica --scenario=E3 --clusterSpeed=5 \
    --nodeSpeed=5 --simTime=200 --RngRun=1"
```

La primera vez, `./ns3 run` compila el programa antes de ejecutarlo. Los parámetros se pasan como `--nombre=valor`; la tabla 1 resume los que se usan. El nombre del escenario es solo una etiqueta: lo que define qué se mueve son las velocidades. Por eso cada escenario se corre con esta combinación, donde v es la velocidad en m/s:

Escenario	`--clusterSpeed`	`--nodeSpeed`
E0 (nada se mueve)	0	0
E1 (solo los nodos)	0	v
E2 (solo los clústeres)	v	0
E3 (ambos)	v	v

Parámetro	**Por defecto**	**Qué hace**
`--scenario`	`E3`	Etiqueta del escenario (E0 a E3); aparece en el nombre del archivo de salida.
`--clusterSpeed`	5	Velocidad de los clústeres (m/s); 0 los deja quietos.
`--nodeSpeed`	5	Velocidad de los nodos dentro de su clúster (m/s); 0 los deja quietos.
`--superSpeed`	0	Velocidad de los superclústeres (m/s); en los experimentos es 0.
`--simTime`	200	Duración de la simulación (s).
`--RngRun`	1	Número de réplica: cada valor da una corrida independiente.
`--outDir`	`resultados/raw`	Carpeta donde se guarda el XML de FlowMonitor; se crea si no existe.
`--netanim`	`false`	Genera además una traza para NetAnim.
`--verificar`	`false`	Corre la prueba de conectividad (paso 2).
`--rangoCluster`	60	Alcance de radio dentro de cada clúster (m).
`--rangoBackbone`	250	Alcance de radio entre CH (m).
`--regionCluster`	150	Semiancho de la región en que se mueve cada clúster (m).
`--regionNodo`	40	Semiancho de la región en que se mueve cada nodo (m).
`--trazaPosiciones`	0	Imprime las posiciones cada tantos segundos (0 = no).

Cuadro 1: Parámetros del programa `manet-jerarquica`.

Al empezar, el programa imprime la configuración de la red (clústeres, nodos, CH, direcciones y posiciones iniciales). Al terminar, guarda el archivo `<escenario>_cs<clusterSpeed>_ns<nodeSpeed>_run<RngRun>.xml` en la carpeta de salida e imprime un resumen con este formato:

```
=== Métricas (FlowMonitor) ===
 Archivo:    <ruta del XML>
 Paquetes:   <recibidos>/<enviados> recibidos
```

```
PDR:        <porcentaje> %
Throughput: <valor> kbit/s
Retardo:    <valor> s
Jitter:     <valor> s
```

Este resumen sirve para comprobar rápido que la corrida funcionó, pero suma todos los flujos, incluidos los mensajes de control de AODV. Las cifras que se reportan se calculan solo sobre los flujos de datos, así que no coinciden con las de la consola.

2.3 Paso 2 (opcional): comprobar que la red está conectada

Para verificar que la red jerárquica está conectada, se corre el programa con la opción `--verificar=true`. Envía paquetes de prueba entre clústeres y termina con el mensaje `RESULTADO: la red jerárquica está conectada.` Si aparece `RESULTADO: hay pares sin conectividad`, hay que revisar la geometría o los alcances. La prueba también confirma que la movilidad configurada no rompe la red en el escenario E0.

2.4 Paso 3: ejecutar los escenarios

El programa se ejecuta una vez por configuración. El escenario se selecciona con `--scenario` y las velocidades con `--clusterSpeed` y `--nodeSpeed`, de acuerdo con la tabla anterior. El control E0 se corre una sola vez, sin movimiento; E1, E2 y E3 se corren a 1, 5, 10 y 20 m/s, y cada configuración se repite diez veces cambiando `--RngRun` para obtener réplicas independientes. En total son 130 corridas. Cada una escribe su propio XML en la carpeta indicada por `--outDir`.

2.5 Paso 4: interpretar la salida

Cada corrida produce un archivo XML de FlowMonitor y un resumen en consola. El XML contiene las estadísticas por flujo: paquetes transmitidos y recibidos, bytes, suma de retardos y suma de variaciones de retardo. A partir de esos contadores se calculan las cuatro métricas del estudio: la tasa de entrega de paquetes (PDR), el retardo extremo a extremo medio, el *throughput* y el *jitter*. Solo se consideran los ocho flujos de datos (puertos 5000 a 5007); el tráfico de control de AODV (puerto 654) se excluye porque no representa carga útil.

El resumen en consola incluye todos los flujos, también los de AODV, por lo que sus cifras no coinciden con las que se obtienen al analizar únicamente los flujos de datos. Para comparar escenarios se usan las medias de las diez réplicas de cada configuración y sus intervalos de confianza del 95 %; la comparación formal entre E2 y E1 se hace con una prueba t de Welch unilateral.

2.6 Visualizar una simulación con NetAnim

Con `--netanim=true`, el programa guarda en la carpeta de salida el archivo `netanim_<escenario>_cs<clusterSpeed>_ns<nodeSpeed>_run<RngRun>.xml`. Al abrirlo con NetAnim, que viene con ns-allinone, se ve el movimiento de los nodos: los CH en rojo y los miembros en azul.

2.7 Problemas frecuentes

Mensaje o síntoma	**Qué hacer**
`./ns3` no arranca con el Python del sistema	Indicar otro intérprete con la variable `NS3_PYTHON`.
Error al compilar el programa	Verificar que ns-3 se configuró con los módulos `wifi`, `aodv`, `mobility`, `internet`, `applications`, `flow-monitor` y `netanim`.
Todo es muy lento en WSL	Mover ns-3 y el proyecto al disco de Linux (`~`) en vez de `/mnt/c/...`.